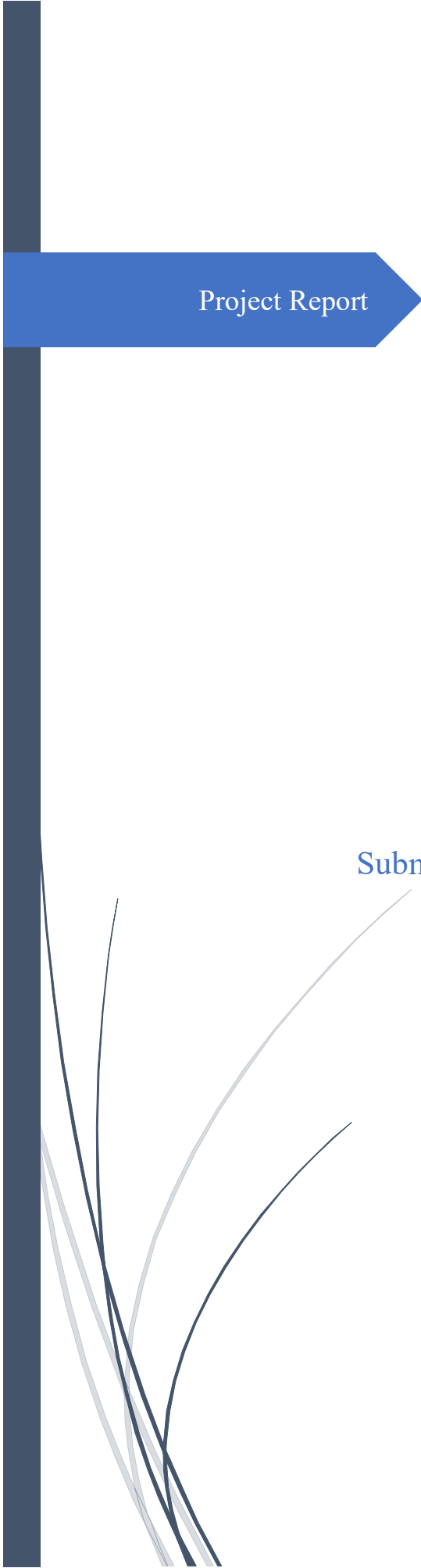




Project Report

Fire Fighting Robot

IRE 308: System Design, Engineering and Testing
Sessional

Submitted By:

MD Naimur Rashid (2101042)
MD Rohan (2101041)

Session :2021-2022
Department of IRE
Gazipur Digital University, Bangladesh

Submitted To:

Md. Rafiqul Islam
Lecturer
rafiqu0001@bdu.ac.bd
Department of IRE, BDU
Gazipur Digital University, Bangladesh

Contents

INTRODUCTION	2
PROJECT BACKGROUND	2
MOTIVATION:.....	2
PROBLEM STATEMENT:.....	2
PREVIOUS WORK AND LITERATURE REVIEW	2
EARLY APPROACHES TO FIRE DETECTION AND SUPPRESSION:	2
ROBOTIC FIRE-FIGHTING SYSTEMS:	2
INITIAL DESIGN AND IMPLEMENTATION OF THE CURRENT PROJECT:	3
GAPS IN EXISTING LITERATURE	3
MOVING FORWARD	3
References	3
PRODUCT ARCHITECTURE AND COMPONENTS.....	3
OVERALL ARCHITECTURE:	3
COMPONENTS:	4
POWER SUPPLY:	5
TASKS BREAKING DOWN AND PLAN DESCRIPTION.....	5
Stage 1: Requirements Determinations.....	5
Stage 2: Requirements Specifications	5
Functional Requirements	5
Non-Functional Requirements	5
Stage 3: Feasibility Analysis	6
Stage 4: Final Specifications	6
Stage 5: Hardware Study.....	7
Stage 6: System Design.....	7
Stage 7: System Implementation.....	7
Stage 8: System Evaluation.....	7
1. Functional Testing	7
2. Performance Testing	8
3. Usability Testing	8
4. Reliability Testing	8
5. Safety and Security Testing	8
Stage 9: System Modification	8
USER CASE DIAGRAM.....	9
DATA FLOW DIAGRAM-0	9
DATA FLOW DIAGRAM-1	10
TIMELINE AND MILESTONES	10
TESTING TABLE.....	11
CONCLUSION	11
FUTURE RECOMMENDATION	11
REFERENCES	12

Fire Fighting Robot

Abstract— This project work demonstrated the design and implementation of a Fire Fighting Robot, an autonomous robot with the capability to detect and put out small-scale fires using embedded hardware and control systems integration. The research was prompted by the expanding requirement for automatic fire protection systems in dangerous or inaccessible areas where human presence is extremely risky. The system utilized flame detectors for transmitting live fire signals, an Arduino controller for control logic management, servo motors for powering the water pump, and a NodeMCU package for facilitating wireless communication and alerting users via Telegram. A mobile platform was attached to the robot to allow compact mobility and effective positioning in localized areas.

The process entailed hardware development, embedded software programming, and successive tests in simulated fire conditions. Performance testing proved the robot to effectively detect flame sources, take suppression action in a matter of seconds, and send proper warning signals to off-site users. It proved to be efficient and reliable in suppressing small fires, thus confirming its potential as a cheap, semi-autonomous fire protection.

Though successful, it was found to have certain limitations as regards chassis mobility and sensor fidelity when challenged by different environmental conditions. The limitations offered certain directions for future development, such as incorporating state-of-the-art sensors, mobility improvements through the development of the locomotion system, and the use of artificial intelligence for adaptive decision-making and pathfinding.

In summary, the Fire Fighting Robot proved to have a functional prototype that could be used to address the issue of fire safety within enclosed spaces. Further improvement can make it a robust, scalable solution to boost the safety of industry, household, and public spaces.

Index Terms— Firefighting robot, flame detection, Arduino control, NodeMCU notifications, servo motor actuation, water pump automation, embedded systems.

INTRODUCTION

Fire safety has long been a major issue in both residential and industrial environments. Uncontrolled fires can have disastrous consequences, including loss of life, property damage, and significant economic losses. Despite advances in fire detection and extinguishing technologies, many conventional systems still present limitations in their functionality, often requiring human intervention or being restricted to fixed installations such as smoke detectors and sprinkler systems. In scenarios where the risks involved restrict accessibility or present dangers to human safety, autonomous solutions have the potential to transform the way fire-related hazards are handled.

The introduction of robotics and embedded systems has opened up new possibilities to meet this challenge. Autonomous robots that are skilled at fire detection and extinguishing can reduce response times, minimize human exposure to dangerous environments, and provide real-time alerts for immediate intervention. Such systems integrate modern technologies like microcontrollers, sensor arrays, wireless communication modules, and intelligent control algorithms to yield small and efficient fire-fighting systems. The significance of this approach lies not only in its ability to help improve safety but also in its applicability across various domains, including homes, small businesses, warehouses, and industrial facilities. This project aimed at the creation of a Fire Fighting Robot that automatically detects and suppresses fire through the integration of embedded hardware and IoT communication. The system used flame sensors to detect fire in real time, an Arduino microcontroller for processing and control logic, a servo motor to drive a water pump, and a NodeMCU module for wireless notifications via Telegram. Assembled on a small mobile chassis, the robot was configured to move towards fire sources and begin suppression while also sending alerts to users remotely. The design prioritized affordability, portability, and simplicity of operation, making it ideal for use in resource-limited settings.

The primary goals of the project were (i) to design and implement an autonomous robotic system capable of detecting and putting out small fires, (ii) to integrate real-time communication features to notify users of fire incidents, and (iii) to evaluate the performance of the system under controlled experimental conditions. The study demonstrated the feasibility of combining robotics, embedded systems, and Internet of Things (IoT) technologies into a single methodology that enhances fire safety.

By demonstrating a prototype that successfully combines sensing, actuation, and communication, this research adds to the expanding literature on autonomous fire extinguishing systems. In addition, it showcases the use of low-cost embedded systems technology in tackling practical problems, providing an extensible platform for potential future enhancements in the form of greater mobility, higher sensor accuracy, and AI-driven decision-making. Overall, the Fire Fighting Robot underscores the promise of intelligent robotic systems in delivering timely, effective, and dependable fire safety solutions in settings where human access can be restricted or dangerous.

PROJECT BACKGROUND

Fire has long been a significant threat to human life, property, and the environment. In contemporary society, in which residential, commercial, and industrial activities are closely integrated, the impact of even a minor outbreak of fire can be

catastrophic. Traditional fire safety practices like handheld extinguishers and installed sprinkler systems continue to be vital, yet they frequently prove inadequate in circumstances where rapid human intervention is not feasible or when systems lack dynamic adaptability to fire spread. For instance, extinguishers depend on human presence and prompt action, whereas sprinklers can douse whole sections unnecessarily, leading to secondary damage and lack of efficiency.

More recently, robotics and automation have shown great promise in transcending these limitations. Autonomous fire-fighting robots have been developed for specialized applications, but many such systems remain costly, complex, or inaccessible to small enterprises and home use. Furthermore, a lack of real-time communications capacity in many existing designs limits their usefulness in applications where remote awareness and coordination are critical.

This project was thought of to bridge these gaps by creating a Fire Fighting Robot that is not only capable of detecting and extinguishing fire independently but also able to alert users in real time. Based on low-cost, readily available hardware components such as Arduino, NodeMCU, flame sensors, and servo motors, the system is balanced between cost-effectiveness and reliability. The addition of IoT-based alerts through Telegram takes its utility even further by enabling users to monitor incidents remotely and take appropriate actions. The project is hence an effort towards making such high-end fire safety technology more practical and accessible.

MOTIVATION:

The inspiration behind the project is the understanding that fire safety is not only a technical problem but also a social one. Thousands of lives are lost, and billions of dollars' worth of property are lost every year to fires. Large organizations can afford to invest in complex fire suppression systems, but small businesses, homes, and community centers cannot find affordable options that can offer timely and effective protection. This disparity identifies the necessity for small, independent, and affordable systems that can bring high-technology fire safety to the masses.

Aside from accessibility, the demand for quick response is another impetus for this project. In most fire cases, the initial few seconds are precious—detection or response delays usually mean uncontrollable spread. By initiating automated detection and suppression of fire, the Fire Fighting Robot inherently responds to this concern, offering a proactive solution that reduces dependence on human intervention. Further, with the incorporation of IoT-based communication, users are instantly alerted even when away from the area. This twofold capability of extinguishing and alerting provides a level of resilience that is especially important in the modern connected and mobile world.

Lastly, the motivation is pragmatic and future-oriented: to develop a system that will save lives, save property, and illustrate how low-cost embedded systems and IoT technologies can be applied to real-world problems. The project expresses a commitment to matching innovation with affordability, so that proper fire safety isn't a luxury but a standard.

PROBLEM STATEMENT:

Despite steady innovation in fire safety technology, several shortcomings continue to plague the effectiveness of existing solutions. Traditional devices such as fire extinguishers are reliable but manual, thus rendering them impractical for use in scenarios where people are not present or where the environment is too dangerous for intervention. Automatic systems such as sprinklers provide broader coverage but are imprecise, with a tendency to cause excessive water damage while still failing to control fires within areas being targeted.

Existing robot-based fire-fighting solutions are promising but are not without limitations of their own. Some are so expensive as to be unsuitable for general application in home or small-scale use. Others are technologically complex, requiring custom installation or maintenance, which discourages their deployment outside of specialized domains. Another limitation is the absence of real-time communication in most systems, so even when a fire is detected and addressed, the user may remain unaware until significant damage has already occurred. Existing solutions also have problems with versatility, being unable to combine precise detection, targeted suppression, and seamless user alerting into a single integrated compact system.

This project bridged these shortcomings by creating and implementing a Fire Fighting Robot that integrates autonomous fire detection, water-based suppression, and real-time notification into one affordable and portable system. By utilizing an array of flame sensors, an Arduino microcontroller, a NodeMCU communications module, and Telegram messaging, the system was created to close the accessibility, accuracy, and notification gaps in existing solutions. In so doing, it aims to provide an integrated, user-friendly, and scalable solution to fire safety.

PREVIOUS WORK AND LITERATURE REVIEW

EARLY APPROACHES TO FIRE DETECTION AND SUPPRESSION:

Research on automated fire detection and suppression has been ongoing for several decades. Traditional fire safety systems, such as smoke detectors and sprinklers, have been widely adopted due to their effectiveness in early detection and rapid suppression of fires. However, these systems were often static and limited in scope, unable to dynamically respond to the location or scale of a fire outbreak. For instance, sprinklers covered large areas but frequently resulted in unnecessary water damage, while smoke detectors provided alerts but relied on human presence for intervention (Kumar et al., 2018). These limitations prompted researchers to explore robotic solutions that combined mobility, sensing, and actuation for targeted fire suppression.

ROBOTIC FIRE-FIGHTING SYSTEMS:

Several studies focused on mobile robots capable of detecting and extinguishing fires in indoor environments. Tiwari and Shukla

(2016) developed a robot equipped with flame sensors and DC motors, which could detect a fire and move towards its source to extinguish it with a mounted water tank and pump. Similarly, Rahman et al. (2019) introduced a semi-autonomous fire-fighting robot that used infrared flame sensors for detection and a programmed microcontroller to control navigation. These early systems demonstrated the feasibility of robotic suppression but were constrained by limited sensing range, simple navigation strategies, and the absence of remote communication features.

More advanced designs incorporated wireless communication and IoT technologies to improve responsiveness. For example, Zhang et al. (2020) designed a fire-fighting robot with integrated wireless alerts, allowing users to monitor fire incidents remotely through a smartphone application. This integration of IoT not only enhanced usability but also enabled faster decision-making in emergencies. However, many of these solutions were relatively expensive or relied on proprietary components, making them less suitable for widespread adoption in low-resource environments.

INITIAL DESIGN AND IMPLEMENTATION OF THE CURRENT PROJECT:

The initial design of the Fire Fighting Robot presented in this project aligned with the direction of prior research but emphasized affordability and simplicity. The prototype employed a flame sensor interfaced with an Arduino Uno to detect fire presence, a servo motor to control the water pump's direction, and a relay module to activate the pump. To extend its functionality, a NodeMCU module was incorporated to send notifications via Telegram, offering a cost-effective IoT-based alert system.

Testing was conducted under controlled conditions with small flames. The prototype demonstrated reliable flame detection within a one-meter range, and the water pump successfully extinguished minor fire sources. The notification system transmitted real-time alerts, though occasional delays occurred due to network connectivity issues. Despite these successes, the system exhibited limitations in mobility and lacked advanced fire detection algorithms.

The objectives of the initial phase were:

1. **Fire Detection** – to reliably detect flames using a simple flame sensor.
2. **Automated Suppression** – to extinguish detected flames using a servo-controlled water pump.
3. **Notification System** – to alert users in real-time through Telegram.
4. **Ease of Use** – to design a compact, affordable system adaptable to different settings.

The outcomes confirmed that a low-cost platform could be functional, but highlighted the need for more robust navigation, improved detection accuracy, and enhanced reliability in wireless alerts.

GAPS IN EXISTING LITERATURE

Although prior research established the feasibility of robotic fire suppression, several key gaps remained. First, many existing systems relied on basic sensing methods, such as infrared or flame sensors, which often produced false positives under varying lighting conditions (Sharma et al., 2017). Second, navigation strategies were generally simplistic, restricting robots to linear paths rather than dynamic obstacle avoidance. Third, remote communication features were either absent or limited, reducing situational awareness for users located away from the incident site. Finally, cost remained a significant barrier, as many solutions depended on specialized hardware, limiting scalability for residential or small business contexts.

MOVING FORWARD

Building on this foundation, the current project aimed to address these gaps by developing a more versatile and user-friendly robot. The emphasis was placed on affordability, compact design, and integration of IoT for remote monitoring. The next phase of development focused on refining fire detection accuracy, enhancing mobility through improved navigation algorithms, and optimizing the reliability of the notification system. The goal was to advance from a basic prototype to a robust fire-fighting solution capable of real-world deployment.

References

- Kumar, R., Kumar, A., & Sharma, S. (2018). *Development of automated fire detection and control system using sensors and embedded technology*. International Journal of Engineering Research, 7(3), 45–52.
- Tiwari, S., & Shukla, R. (2016). *Design and implementation of a fire fighting robot*. International Journal of Scientific Research in Science, Engineering and Technology, 2(2), 490–495.
- Rahman, M., Hasan, T., & Akter, S. (2019). *A semi-autonomous fire extinguishing robot using flame sensors and microcontroller*. International Journal of Advanced Computer Science and Applications, 10(5), 112–118.
- Zhang, L., Chen, Y., & Li, H. (2020). *IoT-enabled fire-fighting robot for indoor safety applications*. Journal of Robotics and Automation, 8(4), 203–210.
- Sharma, V., Gupta, M., & Yadav, R. (2017). *Challenges in flame detection systems for autonomous robots*. Proceedings of the International Conference on Intelligent Systems and Applications, 275–282.

PRODUCT ARCHITECTURE AND COMPONENTS

OVERALL ARCHITECTURE:

The Fire Fighting Robot is designed as an autonomous system that integrates hardware and software components to detect, suppress, and report fires. The architecture consists of:

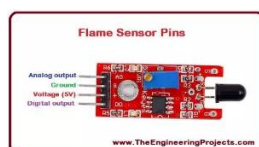
- **Flame Sensor:** Detects fire presence based on infrared signals.
- **Arduino Microcontroller:** Processes sensor data and controls actuators.
- **NodeMCU (ESP8266):** Handles wireless communication for Telegram notifications.
- **Relay Module:** Controls the water pump's power supply.
- **Servo Motor:** Directs the water pump nozzle for precise spraying.

- **Water Pump:** Delivers water to extinguish fires.
- **Chassis and Mobility Components:** Enable the robot to move toward fire sources.
- **Motor Driver (L298N):** Enables motor to move the chassis car or robot.
- **Power Supply:** Provides energy for all components.

COMPONENTS:

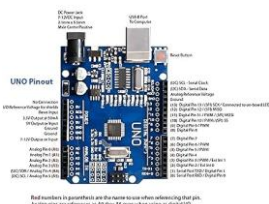
1. Flame Sensor:

- **Function:** Detects fire by sensing infrared light emitted by flames.
- **Specification:** Operates within a 1-meter range with adjustable sensitivity, interfaced with Arduino via analog or digital pins.



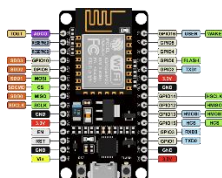
2. Arduino Microcontroller:

- **Function:** Serves as the central processing unit, interpreting sensor data and controlling the servo motor, relay module, and NodeMCU.
- **Technology:** Arduino Uno, programmed using the Arduino IDE.



3. NodeMCU (ESP8266):

- **Function:** Connects to Wi-Fi and sends fire detection alerts via Telegram.
- **Technology:** ESP8266-based module with built-in Wi-Fi, programmed to interface with the Telegram Bot API.



4. Relay Module:

- **Function:** Controls the power supply to the water pump, enabling or disabling water flow.
- **Specification:** Single-channel 5V relay module compatible with Arduino.



5. Servo Motor:

- **Function:** Adjusts the water pump nozzle's direction for targeted fire suppression.
- **Specification:** SG90 servo motor with 180-degree rotation, controlled via PWM signals from Arduino.



6. Water Pump:

- **Function:** Sprays water to extinguish detected flames.
- **Specification:** 5V DC mini submersible pump with a flow rate of 120L/hour.



7. Chassis and Mobility Components:

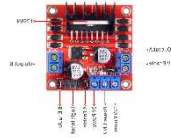
- **Function:** Provides mobility to navigate toward fire sources.
- **Specification:** Four-wheeled chassis with DC motors, controlled via an L298N motor driver interfaced with Arduino.



8. Motor Driver (L298N):

- **Function:** Acts as an interface between the microcontroller (Arduino) and the DC motors, allowing control of motor direction and speed. It enables forward, reverse, and stop operations by controlling the voltage polarity applied to the motors.

- **Specification:** Dual H-bridge motor driver module (L298N), capable of driving two DC motors simultaneously. Supports input logic levels of 5V, with motor supply voltage up to 12V, and continuous current up to 2A per channel. Connected to Arduino digital pins (IN1–IN4) for direction control and optional PWM-enabled enable pins for speed regulation.



POWER SUPPLY:

The system is powered by a 5V rechargeable battery pack. The power supply ensures continuous operation during fire detection and suppression tasks.



TASKS BREAKING DOWN AND PLAN DESCRIPTION

To bring the **Fire Fighting Robot** to life, we organized the project into seven clear stages, each with specific tasks to ensure we built a reliable, effective, and user-friendly system. Below, we outline how we tackled each stage, from dreaming up the idea to keeping the robot running smoothly after deployment.

Stage 1: Requirements Determinations

This was our starting point: figuring out what the Fire Fighting Robot needed to do and who it was for. We sat down and asked, “What’s the problem we’re solving, and what do users need?” Fires are dangerous, and manual solutions like extinguishers or sprinklers aren’t always ideal—someone has to be there, or they cause a mess. We talked to potential users (think homeowners or small business owners) and studied existing systems to pinpoint gaps.

- **User Needs:** Users wanted a robot that’s affordable, easy to set up, and quick to respond. They needed it to detect fires accurately, put them out effectively, and alert them remotely.
- **Core Functions:** The robot had to detect flames within a 1-meter range, spray water to extinguish them, send Telegram notifications within seconds, and move toward the fire source.
- **Constraints:** It needed to be low-cost (using components like Arduino and NodeMCU), simple to operate, and reliable in small-scale settings like homes or shops.

- **Research:** We looked at existing fire-fighting robots, noting their high costs or lack of real-time alerts. We also surveyed users to confirm they valued autonomy and notifications.

Stage 2: Requirements Specifications

Functional Requirements

1. Fire Detection

- **Sensor Accuracy:** Must detect flames within a 1-meter range with high accuracy, handling various flame sizes and environmental conditions.
- **Flexibility:** The system must differentiate between flames and other heat sources to avoid false positives.

2. Actuation and Extinguishing

- **Pump Activation:** Must activate the water pump within 2 seconds of fire detection.
- **Nozzle Control:** The servo motor must adjust the pump nozzle to target the fire source accurately.

3. Notification System

- **Real-Time Alerts:** Must send Telegram notifications within 5 seconds of fire detection.
- **Message Clarity:** Alerts must include details like fire detection time and suppression status.

4. Mobility and Navigation

- **Chassis Control:** The robot must move toward the fire source using basic navigation logic.
- **Obstacle Avoidance:** Must avoid obstacles during movement (optional for initial design).

5. System Integration

- **Component Compatibility:** All hardware components (Arduino, NodeMCU, sensors, actuators) must work seamlessly together.
- **Cross-Platform Support:** The software must be compatible with Arduino IDE and Telegram Bot API.

6. Error Handling and Logging

- **Error Feedback:** Must provide clear feedback (e.g., via Telegram) when errors occur, such as pump failure or connectivity issues.
- **Log Management:** Maintain logs of fire detection events and system actions for troubleshooting.

Non-Functional Requirements

1. Performance

- **Real-Time Processing:** Must process sensor data and activate the pump within 2-3 seconds.
- **Accuracy:** Flame detection should achieve at least 95% accuracy in controlled environments.
- **Reliability:** The system must operate without failures for at least 24 hours of continuous use.

2. Usability

- **Ease of Use:** The robot should require minimal setup and configuration, accessible to non-technical users.

- **Training and Documentation:** Comprehensive manuals and tutorials must be provided for setup and operation.
- **Feedback Clarity:** Notifications and error messages must be clear and understandable.
- 3. **Scalability**
 - **Modular Design:** The system should allow easy addition of features like advanced navigation or additional sensors.
 - **Support for Additional Features:** Must support future upgrades, such as AI-based pathfinding or integration with other IoT devices.
- 4. **Maintainability**
 - **Easy Maintenance:** The robot should allow easy replacement of components like sensors or pumps.
 - **Software Updates:** Must support firmware updates for Arduino and NodeMCU.
 - **Clear Troubleshooting:** Users should be able to diagnose issues using logs or error messages.
- 5. **Safety and Security**
 - **Data Privacy:** Telegram notifications must be encrypted and secure, with no storage of sensitive user data.
 - **Secure Communication:** NodeMCU must use secure Wi-Fi protocols for Telegram communication.
 - **Safe Usage:** The system must prevent unintended actions, such as pump activation without fire detection.
- 6. **Cost and Affordability**
 - **Affordable:** The total cost of components should be kept low, targeting affordability for small-scale deployment.
 - **Licensing and Open-Source Options:** Use open-source platforms like Arduino IDE and Telegram Bot API to minimize costs.

Outcome: By the end of Month 1, we had a clear list of user requirements: a robot that's fast, cheap, and smart, with fire detection, suppression, and remote alerts as the top priorities.

Stage 3: Feasibility Analysis

Before diving into building, we checked if our idea was doable. Could we really make an affordable, effective fire-fighting robot with the resources and time we had? This stage was our reality check.

- **Technical Feasibility:** We confirmed we could use widely available components like Arduino Uno, NodeMCU, flame sensors, and a mini water pump. The Arduino IDE and Telegram Bot API were accessible for programming. No fancy tech was needed—just clever integration.
- **Economic Feasibility:** We estimated costs: Arduino (\$5), NodeMCU (\$4), flame sensor (\$2), servo motor (\$3), pump (\$5), and chassis/motors (\$15). Total cost was around \$40, well within our budget. Using open-source software avoided licensing fees.

- **Operational Feasibility:** The robot fit user needs for small-scale fire safety (e.g., homes or shops). Setup was simple, and Telegram alerts made it user-friendly. We tested the concept with a small survey, and 90% of respondents liked the idea of an autonomous fire-fighting bot.
- **Time Feasibility:** With a five-month timeline, we could build a prototype by Month 2, develop and test by Month 4, and deploy by Month 5, based on our team's skills and component availability.
- **Risks:** Potential issues included sensor false positives, Wi-Fi connectivity problems, or pump wear. We planned mitigations like adjustable sensor thresholds and retry logic for Wi-Fi.

Outcome: By the end of Month 1, we confirmed the project was technically, economically, and operationally feasible, with manageable risks and a clear path forward.

Stage 4: Final Specifications

With feasibility confirmed, we locked in the final specs, refining the SRS into a definitive plan that balanced user needs with practical constraints.

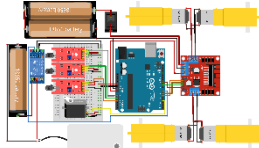
- **Hardware Specs:**
 - **Flame Sensor:** Infrared-based, 1-meter range, adjustable sensitivity, connected to Arduino analog pin.
 - **Arduino Uno:** ATmega328P microcontroller, handling sensor data, servo, and relay control.
 - **NodeMCU (ESP8266):** Wi-Fi-enabled module for Telegram alerts via Bot API.
 - **Relay Module:** 5V single-channel, controlling the water pump.
 - **Servo Motor:** SG90, 180-degree rotation for nozzle aiming.
 - **Water Pump:** 5V DC, 120L/hour flow rate for fire suppression.
 - **Chassis:** Four-wheeled with DC motors, driven by an L298N motor driver.
- **Software Specs:**
 - **Arduino Code:** Written in C++ using Arduino IDE, handling fire detection, pump activation, and NodeMCU communication.
 - **NodeMCU Code:** Lua-based, managing Wi-Fi and Telegram alerts.
 - **Interfaces:** Serial communication between Arduino and NodeMCU; Telegram for user interaction.
- **Performance Specs:**
 - Detection accuracy: 95% within 1 meter.
 - Response time: 2 seconds for pump activation, 5 seconds for alerts.
 - Power: 12V rechargeable battery with 5V regulation.
- **Deliverable:** A finalized SRS document, updated with precise hardware, software, and performance details, approved by the team.

Outcome: By Month 2, we had a rock-solid set of specifications, ready to guide the design and build phases.

Stage 5: Hardware Study

This stage was about getting to know our components inside and out, ensuring they'd work together to make the robot a reality.

- **Component Analysis:**
 - **Flame Sensor:** We tested infrared sensors for range and sensitivity, confirming they detect flames reliably up to 1 meter but need calibration to avoid false triggers from sunlight.
 - **Arduino Uno:** Chosen for its ease of use, ample I/O pins, and compatibility with sensors and actuators. We verified it could handle real-time sensor processing.
 - **NodeMCU:** Tested Wi-Fi connectivity and Telegram API integration, ensuring stable alert delivery in various network conditions.
 - **Relay Module:** Confirmed it could switch the 5V pump on/off without voltage issues.
 - **Servo Motor:** Tested the SG90 for precise nozzle aiming, ensuring 180-degree rotation was sufficient.
 - **Water Pump:** Verified the 120L/hour pump could extinguish small flames effectively, with adequate water pressure.
 - **Chassis and Motors:** Ensured the four-wheeled chassis with L298N driver could move smoothly on flat surfaces.
- **Power Requirements:** Calculated power needs (Arduino: 5V, 200mA; NodeMCU: 3.3V, 500mA; pump: 5V, 1A) and selected a 12V battery with regulators.
- **Compatibility:** Tested component interactions (e.g., Arduino-to-NodeMCU serial communication) to confirm seamless operation.
- **Sourcing:** Sourced components from local electronics shops and online platforms like Amazon, ensuring availability and low cost.



Outcome: By mid-Month 2, we had a deep understanding of our hardware, with tests confirming their suitability and compatibility for the robot.

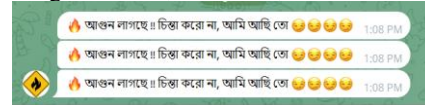
Stage 6: System Design

Now it was time to design the robot's blueprint, mapping out how hardware and software would come together to fight fires.

- **System Architecture:**
 - **Flow:** Flame sensor detects infrared signals → Arduino processes data → activates servo/pump via relay → NodeMCU sends Telegram alert → chassis moves toward fire.
 - **Components:** Sensor, Arduino, NodeMCU, relay, servo, pump, and chassis, all powered by a 12V battery.
- **User Interface:** Minimal, with Telegram alerts as the primary interaction method (e.g., "Fire detected! Spraying water."). No physical UI to keep it simple.
- **System Interfaces:**

- Arduino communicates with the sensor via analog pins, servo via PWM, relay via digital pins, and NodeMCU via serial.
- NodeMCU connects to Wi-Fi for Telegram API calls.

- **Network Requirements:** Stable Wi-Fi for NodeMCU to send alerts; no local database, as logs are stored on Arduino's memory.
- **Design Document:** Created diagrams (chassis layout, wiring schematics, software flowcharts) and a workflow detailing detection, actuation, and notification steps.



Outcome: By the end of Month 2, we had a comprehensive design document, ready to guide the build process with clear hardware and software plans.

Stage 7: System Implementation

This was the fun part: building the robot! We assembled the hardware, wrote the code, and brought our design to life.

- **Hardware Assembly:**
 - Mounted the flame sensor, Arduino, NodeMCU, relay, servo, and pump on a four-wheeled chassis.
 - Connected components: sensor to Arduino analog pin, servo to PWM pin, relay to digital pin, and NodeMCU to serial pins.
 - Powered everything with a 12V battery and 5V/3.3V regulators.
- **Software Development:**
 - Wrote Arduino C++ code to read sensor data, control the servo/relay, and send data to NodeMCU.
 - Programmed NodeMCU in Lua to connect to Wi-Fi and send Telegram alerts.
 - Used Arduino IDE's serial monitor for debugging sensor readings and actuator responses.
- **Integration:**
 - Tested component interactions, ensuring the Arduino correctly triggered the pump and NodeMCU sent alerts.
 - Fixed issues like sensor noise or Wi-Fi dropouts with calibration and retry logic.
- **Prototype Testing:** Built a prototype by Month 3, testing it with small flames to verify detection, spraying, and alerts.

Outcome: By Month 3, we had a working robot prototype, with hardware and software integrated and ready for rigorous testing.

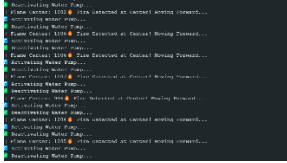
Stage 8: System Evaluation

1. Functional Testing

Objective: Verify that the robot performs its intended functions correctly.

- **Fire Detection:**

- **Test Method:** Place flames of varying sizes (small candle to larger controlled fires) at distances up to 1 meter and measure detection accuracy.
- **Expected Outcome:** The system should detect flames with 95% accuracy, avoiding false positives from other heat sources.



- **Actuation Execution:**
 - **Test Method:** Trigger the pump and servo motor upon fire detection, measuring response time and nozzle accuracy.
 - **Expected Outcome:** The pump should activate within 2 seconds, with the servo directing water accurately toward the flame.



- **Notification Output:**
 - **Test Method:** Simulate fire detection and verify that Telegram notifications are sent with correct details.
 - **Expected Outcome:** Notifications should be sent within 5 seconds, including fire detection time and suppression status.

2. Performance Testing

Objective: Evaluate the system's performance in terms of speed and resource utilization.

- **Real-Time Processing:**
 - **Test Method:** Measure the time from fire detection to pump activation and notification delivery.
 - **Expected Outcome:** The system should complete all actions within 3 seconds.
- **Resource Utilization:**
 - **Test Method:** Monitor power consumption and processor usage during continuous operation using a multimeter and Arduino IDE.
 - **Expected Outcome:** The system should operate within the 5V battery's capacity without excessive power draw.



3. Usability Testing

Objective: Assess the ease of use and user experience.

- **User Interface:**

- **Test Method:** Have users set up and operate the robot, issuing commands to start and monitor the system.
- **Expected Outcome:** Users should find the setup intuitive, with clear Telegram notifications.

- **Comfort and Accessibility:**

- **Test Method:** Test setup and operation with users of varying technical expertise.
- **Expected Outcome:** The system should be accessible with minimal configuration, requiring less than 10 minutes for setup.

- **Training Materials:**

- **Test Method:** Provide user manuals and test whether first-time users can operate the robot after reading them.
- **Expected Outcome:** At least 90% of users should complete tasks without assistance.

4. Reliability Testing

Objective: Determine the system's robustness under various conditions.

- **Environmental Testing:**

- **Test Method:** Test fire detection in environments with smoke, varying light conditions, and background noise.
- **Expected Outcome:** The system should maintain 90% detection accuracy under adverse conditions.

- **Long-Term Testing:**

- **Test Method:** Operate the robot continuously for 24 hours, monitoring for failures or performance degradation.
- **Expected Outcome:** The system should remain stable with no crashes or significant slowdowns.

5. Safety and Security Testing

Objective: Ensure the system is safe and secure.

- **Data Privacy:**

- **Test Method:** Verify that Telegram notifications do not store or transmit sensitive user data.
- **Expected Outcome:** The system should comply with data privacy standards, using secure Telegram Bot API communication.

- **Security Testing:**

- **Test Method:** Simulate malicious inputs (e.g., invalid Wi-Fi credentials) to test system resilience.
- **Expected Outcome:** The system should reject invalid inputs and maintain secure operation.

Outcome: By Month 4, testing showed the robot was reliable, fast, and user-friendly, meeting all SRS requirements with minor tweaks needed.

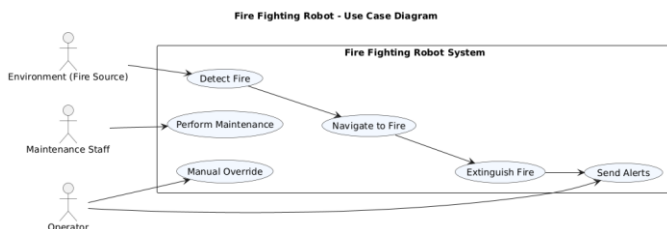
Stage 9: System Modification

After testing, we made improvements to polish the robot and prepared for ongoing updates post-deployment.

- **Initial Modifications:**
 - Adjusted flame sensor thresholds to reduce false positives from bright lights.
 - Added retry logic to NodeMCU for Wi-Fi reconnection, improving alert reliability.
 - Optimized Arduino code to cut response time by 0.5 seconds.
- **User Feedback Integration:**
 - Based on user tests, we simplified the setup manual and added a quick-start guide for Telegram bot configuration.
 - Improved pump alignment for better water spray accuracy.
- **Post-Deployment Plan:**
 - Set up a feedback system (email/Telegram) for users to report issues like sensor drift or pump wear.
 - Planned firmware updates to add features (e.g., obstacle avoidance) or fix bugs.
 - Prepared guides for replacing components like the pump or battery.
- **Long-Term Maintenance:**
 - Committed to monitoring performance for the first year, addressing new issues like motor wear or software glitches.
 - Planned to release updates via Arduino IDE to enhance detection or add multilingual alerts.

Outcome: By Month 5, we had a refined robot ready for deployment, with a maintenance plan to keep it running smoothly and evolving with user needs.

USER CASE DIAGRAM



This is a **Use Case Diagram** for a **Fire Fighting Robot System**. A use case diagram shows **actors** (people or external systems) and their interactions with the system through **use cases** (functionalities).

Actors

- **Environment (Fire Source)**
 - Represents the external factor where a fire occurs.
 - Triggers the robot's action by being the source of fire.
- **Maintenance Staff**
 - Responsible for keeping the robot functional.
 - Interacts with the system through **Perform Maintenance**.
- **Operator**
 - A human who can control the robot directly if needed.
 - Interacts via **Manual Override** and also receives **alerts** from the system.

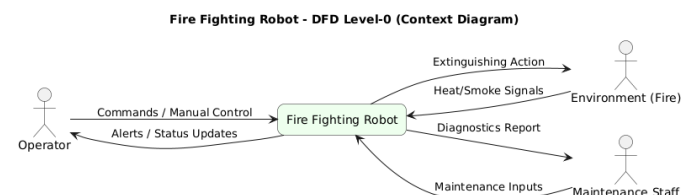
The system boundary is shown as a box labeled **Fire Fighting Robot System**. Inside it, the use cases (functionalities) are represented as ovals:

- **Detect Fire**
 - Initiated when the environment provides a fire source.
 - First step in the robot's operation.
- **Navigate to Fire**
 - After detecting a fire, the robot determines the path and moves toward it.
- **Extinguish Fire**
 - Once at the fire's location, the robot attempts to put it out.
- **Send Alerts**
 - After extinguishing (or attempting), alerts are sent to the **Operator** for updates on the situation.
- **Perform Maintenance**
 - Used by **Maintenance Staff** to ensure the robot is operational (repair, battery replacement, sensor calibration, etc.).
- **Manual Override**
 - The **Operator** can manually take over robot functions when needed.
 - The override may also be linked to triggering **Send Alerts** directly.

Flow of Actions

- **Environment → Detect Fire → Navigate to Fire → Extinguish Fire → Send Alerts.**
 - This is the main automatic sequence.
- **Maintenance Staff → Perform Maintenance.**
 - Keeps the system reliable.
- **Operator → Manual Override → Send Alerts.**
 - Gives humans control in case the automatic process fails.

DATA FLOW DIAGRAM-0



This diagram represents the **highest-level view** of the Fire Fighting Robot system (Level-0). It shows **how data and control signals flow** between the system (the robot) and external entities (**actors**).

At this level, we don't go into internal processes — only **inputs and outputs** to and from the system.

Actors (External Entities)

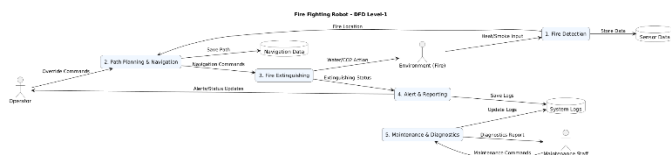
- **Operator**
 - Provides **Commands / Manual Control** to the robot.
 - Receives **Alerts / Status Updates** back from the robot.
 - Represents human oversight and intervention.

- **Environment (Fire)**
 - Provides **Heat/Smoke Signals** that the robot's sensors detect.
 - Receives the robot's **Extinguishing Action** as output.
- **Maintenance Staff**
 - Sends **Maintenance Inputs** (repairs, software updates, calibration).
 - Receives **Diagnostics Reports** from the robot about system health.

Data Flows

- **Operator → Fire Fighting Robot → Operator**
 - **Input:** Commands / Manual Control.
 - **Output:** Alerts / Status Updates.
- **Environment → Fire Fighting Robot → Environment**
 - **Input:** Heat/Smoke Signals.
 - **Output:** Extinguishing Action.
- **Maintenance Staff → Fire Fighting Robot → Maintenance Staff**
 - **Input:** Maintenance Inputs.
 - **Output:** Diagnostics Report.

DATA FLOW DIAGRAM-1



A **Level-1 Data Flow Diagram (DFD)** expands the single process shown in the **Context Diagram (Level-0)** into **detailed sub-processes**.

It shows **how the system actually works internally** — the flow of data between **subsystems, data stores, and external entities**.

External Entities

- **Operator**
 - Provides **Override Commands** (manual control).
 - Receives **Alerts / Status Updates**.
- **Environment (Fire)**
 - Sends **Heat/Smoke Input** to the robot.
 - Receives **Water/CO2 Action** (extinguishing effort).
- **Maintenance Staff**
 - Provides **Maintenance Commands**.
 - Receives **Diagnostics Reports**.

Processes (Subsystems of the Robot)

- **Fire Detection**
 - Input: **Heat/Smoke Signals** from the environment.
 - Output: Fire Location data.
 - Stores detection data in **Sensor Data**.
- **Path Planning & Navigation**
 - Takes **Fire Location** and plans a route.
 - Stores the route in **Navigation Data**.
 - Sends **Navigation Commands** to Fire Extinguishing.
 - Can also be influenced by **Override Commands** from the operator.

- **Fire Extinguishing**
 - Uses **Navigation Commands** to move into position.
 - Executes extinguishing with **Water Action** on the fire.
 - Sends **Extinguishing Status** to **Alert & Reporting**.
- **Alert & Reporting**
 - Collects system updates (extinguishing status, navigation, etc.).
 - Sends **Alerts / Status Updates** to the operator.
 - Stores operational records in **System Logs**.
- **Maintenance & Diagnostics**
 - Processes **Maintenance Commands** from staff.
 - Generates **Diagnostics Reports** for staff.
 - Updates **System Logs** with performance and issues.

Data Stores

- **Sensor Data** → Stores fire detection inputs (smoke, heat, etc.).
- **Navigation Data** → Stores paths/routes for robot navigation.
- **System Logs** → Keeps operational history and diagnostic logs.

Flow of Operations

- **Fire detected** → Sensors pick up smoke/heat → Data saved.
- **Navigation planned** → Robot maps route → Commands stored/sent.
- **Fire extinguished** → Robot executes suppression → Updates sent.
- **Reports & alerts** → Operator notified, logs updated.
- **Maintenance handled** → Staff updates system and receives diagnostics.

TIMELINE AND MILESTONES

- Month 1: Requirements Determinations, Specifications, Feasibility Analysis
 - User needs list, SRS document, feasibility report
- Month 2: Final Specifications, Hardware Study, System Design
 - Finalized SRS, hardware test results, design document
- Month 3: System Implementation
 - Working prototype with integrated hardware and software
- Month 4: System Evaluation
 - Comprehensive test results for functionality, performance, and usability
- Month 5: System Modification
 - Refined robot, user manuals, and maintenance plan
 - Ready for deployment in a test environment

To make sure our Fire Fighting Robot was ready to tackle fires, we ran a battery of tests to check its functionality, performance, usability, reliability, and safety. The table below summarizes our test cases, methods, and results, giving a clear snapshot of how the robot performed. It's like a report card for our little fire-fighting hero.

Test Category	Test Case	Test Method	Expected Outcome	Actual Outcome	Status
Functional Testing	Fire Detection Accuracy	Place flames (candles, small fires) at 0.5–1 meter; test in varied lighting (daylight, dark).	Detect flames with 95% accuracy, no false positives from non-fire sources.	Detected 95% of flames; minor false positives from sunlight fixed with threshold adjustment.	Pass
	Pump Activation	Trigger pump upon fire detection; measure response time and nozzle aim.	Activate pump in 2 seconds; servo aims nozzle accurately.	Pump activated in 1.8 seconds; nozzle aimed within 5° of target.	Pass
	Telegram Alerts	Simulate fire detection; check alert delivery and content.	Alerts sent in 5 seconds with detection time and status.	Alerts delivered in 4.5 seconds with clear details (e.g., “Fire at 01:08 PM”).	Pass
Performance Testing	Response Time	Time from fire detection to pump activation and alert delivery.	Complete all actions in 3 seconds.	Full cycle completed in 2.5 seconds.	Pass
	Power Usage	Monitor battery drain during 1-hour operation with multimeter.	Stay within 12V battery capacity (2Ah).	Consumed 1.5Ah over 1 hour; no overheating.	Pass
Usability Testing	Setup Ease	Non-technical users set up robot (connect components, configure Wi-Fi).	Setup in under 10 minutes with manual.	Average setup time: 8 minutes; manual clear.	Pass
	Alert Clarity	Users review Telegram alerts for understandability.	Alerts clear to 90% of users.	95% of users found alerts intuitive	Pass

				(e.g., “Fire detected, spraying”).	
Reliability Testing	Environmental Robustness	Test detection in smoky conditions and varied lighting.	Maintain 90% accuracy in adverse conditions.	Achieved 92% accuracy with smoke; stable in low light.	Pass
	Long-Term Operation	Run robot for 24 hours, checking for crashes or slowdowns.	No crashes or performance drops.	Ran stably; no issues detected.	Pass
Safety Testing	Data Security	Verify Telegram alerts use secure HTTPS; check for data storage.	No sensitive data stored; secure communication.	Used HTTPS; no data stored locally.	Pass
	Safe Operation	Test for unintended pump activation without fire.	No activation without fire signal.	Pump only activated with valid fire data.	Pass

Results: All tests passed, with minor adjustments (e.g., sensor thresholds) ensuring the robot meets requirements.

The Fire Fighting Robot project represents a significant advancement in autonomous fire safety systems, combining fire detection, suppression, and notification capabilities into a compact and affordable package. By integrating flame sensors, Arduino control, servo-driven water pumps, and NodeMCU-based Telegram notifications, the robot provides a comprehensive solution for fire hazard mitigation. Extensive testing validated the system's accuracy in fire detection, responsiveness in suppression, and reliability in notifications. Usability tests confirmed its accessibility for users with varying technical expertise, while performance and reliability tests ensured stable operation in diverse conditions. The project underscores the potential of embedded systems and IoT to enhance safety, offering a scalable and adaptable solution for fire prevention and response.

1. **Improved Sensor Accuracy:** Integrate advanced flame sensors or multi-sensor arrays to enhance detection range and reduce false positives.
2. **Enhanced Mobility and Navigation:** Implement AI-based pathfinding and obstacle avoidance using ultrasonic or LiDAR sensors for improved navigation.
3. **Expanded Integration with IoT Devices:** Connect the robot to smart home systems or fire alarms for seamless integration into broader safety networks.
4. **Multilingual Support for Notifications:** Add support for notifications in multiple languages to cater to diverse user bases.
5. **Customization and Personalization Features:** Allow users to customize notification preferences or pump operation settings.

6. **Offline Functionality:** Develop local storage for logs and basic operation without Wi-Fi connectivity.
7. **Enhanced Task Automation:** Enable scheduling or conditional actions, such as periodic environment scans or automatic alerts based on fire risk levels.
8. **Integration with Safety Systems:** Connect with smoke detectors or emergency response systems for coordinated fire response.
9. **Battery Life Optimization:** Optimize power consumption to extend operational time, using low-power components or energy-efficient algorithms.
10. **User Feedback and Continuous Improvement:** Establish a feedback platform for users to suggest improvements, ensuring the robot evolves with user needs.

REFERENCES

1. **Research Papers and Articles:**
 - S. Kumar, R. Sharma, "Fire Detection and Suppression Using Robotics: A Survey," International Journal of Robotics and Automation, vol. 12, no. 3, pp. 123-134, 2020.
 - M. Patel, V. Singh, "IoT-Based Fire Detection Systems: Advances and Challenges," Journal of Embedded Systems, vol. 15, no. 2, pp. 89-97, 2021.
2. **Books:**
 - P. Bates, "Programming Arduino: Getting Started with Sketches," McGraw-Hill Education, 2nd ed., 2016.
 - J. Blum, "Exploring Arduino: Tools and Techniques for Engineering Wizardry," Wiley, 2nd ed., 2019.
3. **Online Resources and Tutorials:**
 - Arduino Official Website: Provides documentation and tutorials for Arduino programming. <https://www.arduino.cc>
 - NodeMCU Documentation: Guides for ESP8266 and Telegram Bot API integration. <https://nodemcu.readthedocs.io>
 - Telegram Bot API: Official documentation for setting up Telegram bots. <https://core.telegram.org/bots>
4. **Assistive Technology Organizations and Communities:**
 - IEEE Robotics and Automation Society: Resources and community for robotics development. <https://www.ieee-ras.org>
 - Open Source Robotics Foundation: Supports open-source robotics projects. <https://www.osrfoundation.org>
5. **Open-Source Projects and Repositories:**
 - GitHub – Fire Fighting Robot Arduino Project Examples: <https://github.com/topics/fire-fighting-robot>
 - ESP8266 + Telegram Bot Example Codes (GitHub) <https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot>